S.No.	Date	Title	Page No.	Teacher's Sign
1.		**Assignment No. 1: Introduction to JDBC** **Set A** **Q1**. Write a java program to count number of records in a table. **Q2.** Write a java program to display all the EName from Emp table. Assume Emp (ENo, EName, Salary) table is already created. **Q3.** Write a java program to create Student table with attributes Rno, Sname, Per. **Set B** **Q1.** Write a java program to accept the details of Hospital (HId, HName, Address, PH_No) and store it into the database. (Use Swing). **Q2**. Write a java program to display the details of Doctor (DNO, DName, Specialization) on JTable. Assume Doctor table is already created. **Q3**. Write a java program to make the changes in data which is in ResultSet if you make the changes in data in database. **Set C** **Q1.** Write a java program for the following: a. Create a table. b. Insert c. Update d. Search e. Display **Q2.** Write a java program to update the salary of given employee and display updated details in JTable. (Use Swing). Assume Emp (ENo, EName, Sal) table is already created. **Q3.** Write a java program for the implementation of Scrollable ResultSet. Consider Teacher (TID, TName, Subject) table is already created.		
2.		**Assignment No 2: MultiThreading** **Set A** **Q1.** Write a multithreading program in java to display all the integers between 1 to 100 randomly after 2 seconds. **Q2.** Write a multithreading program in java to display all the characters between Z to A after 5 seconds. **Q3.** Write a java program to print "Hello Java" message 10 times. **Set B** **Q1.** Program to define a thread for printing text on output screen for 'n' number of times. Create 3 threads and run them. Pass the text 'n' parameters to the thread constructor. Example: i. First thread prints "Hello" 10 times. ii. Second thread prints "Good Morning" 20 times iii. Third thread prints "Sir/Mam" 30 times. **Q2.** Write a java program in multithreading to create 3 balls and bounce them vertically.		

S.No.	Date	Title	Page No.	Teacher's Sign
2.		**Q3.** Write a java program in multithreading to draw five lines vertically. **Set C** **Q1**. Write a program to solve producer consumer problem in which a producer produces a value and consumer consume the value before producer generate the next value. (Hint: use thread synchronization) **Q2.** Write a program to calculate the sum and average of an array of 1000 integers (generated randomly) using 10 threads. Each thread calculates the sum of 100 integers. Use these values to calculate average. [Use join method]. **Q3.** Write a program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.		
3.		**Assignment 3: Socket Programming** **Set A** **Q1.** Write a java program to display IP address of a machine. **Q2.** Write a java program to accept a number in client application, send it to the server, server will check whetherit is Armstrong or not and sends result accordingly to the client. **Q3.** Write a java program to send "Hi" message to the Server. **Set B** **Q1.** Write a java program for standalone chatting application. **Q2.** Write a java program to accept a filename in client application and check whether it is available on server machine or not, if it there then display its contents on client's terminal otherwise display the message "File Not Found". (Write Client and Server application on different terminals) **Q3.** Write a java program to accept a number in client application, send it to the server, server will calculates its factors and display them on the client terminal. (Each factor should display after 2 seconds). Write Client and Server application on different terminals. **Set C** **Q1.** Write a java program for chatting application. (Use Swing for GUI). (WriteClient and Server application on different terminals). **Q2.** Write a java program to send and receive the messages from multiple terminals. **Q3.** Write a java program to share a file on server machine between multiple clients.		

S.No.	Date	Title	Page No.	Teacher's Sign
4.		**Assignment No. 4: Servlet And JSP** **Set A** **Q1**. Design a servlet that provides information about a HTTP request from a client, such as IP address and browser type. The servlet also provides information about the server on which the servlet is running, such as the operating system type, and the names of currently loaded servlets. **Q2.** Write a Program to make use of following JSP implicit objects: i. out: To display current Date and Time. ii. request: To get header information. iii. response: To Add Cookie iv. config: get the parameters value defined in v. application: get the parameter value defined in vi. session: Display Current Session ID vii. pageContext: To set and get the attributes. viii. page: get the name of Generated Servlet **Q3.** Write a Servlet application to display Hello Java Message on the Browser. Q4. Write a JSP script to display all the prime numbers between 1 to n in Red Color. Q5. Write a JSP application to accept the details of Emp (Eno, EName, Salary) and display it in Tabular format on the browser. **Set B** **Q1.** Design an HTML page which passes customer number to a search servlet. The servlet searches for the customer number in a database (customer table) and returns customer details if found the number otherwise display error message. **Q2**. . Design an HTML page containing option buttons (Maths, Physics, Chemistry and Biology) and buttons submit and reset. When the user clicks submit, the server responds by adding a cookie containing the selected subject and sends a message back to the client. Program should not allow duplicate cookies to be written. **Q3**. Write a Servlet program to display the details of PATIENT (PatientNo, PatientName, PatientAddress, Patientage, PatientDiease) in tabular form on browser. Q4.Write a JSP application to accept a user name and greets to the user according to the system Time. Q5. Write a Servlet program for the implementation of session tracking **Set C** **Q1.** Create a JSP page for an online multiple choice test. The questions are randomly selected from a database and displayed on the screen. The choices are displayed using radio buttons. When the user clicks on next, the next question is displayed. When the user clicks on submit, display the total score on the screen. **Q2.** Write a JSP application to accept the details of Teacher (TID, TName, Salary) and display it in tabular format.		

S.No	Date	Title	Page No.	Teacher's Sign
		Q3.Write a java program to share a file on server machine between multiple clients.		

Assignment 3: Socket Programming

Set A

Q1. Write a java program to display IP address of a machine.

```
import java.net.*;

public class IPAdd {

  public static void main(String[] args) {

    try {

    Socket socket = new Socket("localhost" , 80);

    String localIP = socket.getLocalAddress().getHostAddress();

    System.out.println("Local IP Address: " + localIP);

    socket.close();

} catch (Exception e) {

e.printStackTrace();

}

}

}
```

Output:

```
C:\Users\lenovo>javac IPAdd.java

C:\Users\lenovo>java IPAdd
Local IP Address: 192.168.163.185
```

Q2. Write a java program to accept a number in client application, send it to the server, server will check whether it is Armstrong or not and sends result accordingly to the client.

```
//Client side
import java.net.*;
import java.io.*;
import java.util.Scanner;
  public class ArmstrongClient {
  public static void main(String[] args) {
  try (Socket socket = new Socket("localhost", 1234)) {
  System.out.println("Connected to server.");

// Set up input/output streams

DataInputStream inputStream = new DataInputStream(socket.getInputStream());
DataOutputStream outputStream = new DataOutputStream(socket.getOutputStream());
// Get a number from the user
Scanner scanner = new Scanner(System.in);
System.out.print("Enter a number: ");
int number = scanner.nextInt();
// Send the number to the server
outputStream.writeInt(number);
// Receive the result from the server
String result = inputStream.readUTF();
// Print the result
System.out.println("Result from server: " + result);
} catch (IOException e) {
System.err.println("Error connecting to server: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac ArmstrongClient.java

C:\socket>java ArmstrongClient
Connected to server.
Enter a number: 153
Result from server: Armstrong
```

```
//Server side
import java.net.*;
import java.io.*;
public class ArmstrongServer {
public static void main(String[] args) {
try (ServerSocket serverSocket = new ServerSocket(1234)) {
System.out.println("Server is listening on port 1234...");
// Accept client connection
try (Socket socket = serverSocket.accept()) {
System.out.println("Client connected.");
```

```
// Set up input/output streams
DataInputStream inputStream = new DataInputStream(socket.getInputStream());
DataOutputStream outputStream = new DataOutputStream(socket.getOutputStream());
// Read the number sent by the client
int number = inputStream.readInt();
System.out.println("Received number: " + number);
// Check if the number is an Armstrong number
boolean isArmstrong = isArmstrongNumber(number);
// Send the result back to the client
outputStream.writeUTF(isArmstrong ? "Armstrong" : "Not Armstrong");
} catch (IOException e) {
System.err.println("Error handling client: " + e.getMessage());
}
} catch (IOException e) {
System.err.println("Error starting server: " + e.getMessage());
}
}
// Method to check if a number is Armstrong
public static boolean isArmstrongNumber(int number) {
int sum = 0;
int temp = number;
int digits = String.valueOf(number).length();
while (temp != 0) {
int digit = temp % 10;
sum += Math.pow(digit, digits);
temp /= 10;
}
return sum == number;
}
}
```

Output:

```
C:\socket>javac ArmstrongServer.java

C:\socket>java ArmstrongServer
Server is listening on port 1234...
Client connected.
Received number: 153
```

Q3. Write a java program to send “Hi” message to the Server.

```
//Server side

import java.net.*;
import java.io.*;
public class ServerHi {
public static void main(String[] args) {
try (ServerSocket serverSocket = new ServerSocket(1234)) {
System.out.println("Server is listening on port 1234...");
try (Socket socket = serverSocket.accept()) {
System.out.println("Client connected.");
BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
String message = input.readLine();
System.out.println("Received message: " + message);
}
} catch (IOException e) {
System.err.println("Error in server: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac ServerHi.java

C:\socket>java ServerHi
Server is listening on port 1234...
Client connected.
Received message: Hi
```

```
//Client side
import java.net.*;
import java.io.*;
public class ClientHi {
public static void main(String[] args) {
try (Socket socket = new Socket("localhost", 1234)) {
System.out.println("Connected to server.");
PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
output.println("Hi");
System.out.println("Message sent: Hi");
} catch (IOException e) {
System.err.println("Error in client: " + e.getMessage());
}
}
}
```

Output:

```
C:\socket>javac ClientHi.java

C:\socket>java ClientHi
Connected to server.
Message sent: Hi
```

Set B

Q1. Write a java program for standalone chatting application.

```
//Server side

import java.net.*;
import java.io.*;
    public class ChatServer {
    public static void main(String[] args) {
    try (ServerSocket serverSocket = new ServerSocket(1234)) {
      System.out.println("Server is listening on port 1234...");
    try (Socket socket = serverSocket.accept()) {
      System.out.println("Client connected.");
       BufferedReader input = new BufferedReader(new
      InputStreamReader(socket.getInputStream()));
      PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
String clientMessage;
BufferedReader consoleInput = new BufferedReader(new InputStreamReader(System.in));
while (true) {
clientMessage = input.readLine();
if (clientMessage.equalsIgnoreCase("exit")) {
System.out.println("Client disconnected.");
break;
}
System.out.println("Client: " + clientMessage);
System.out.print("Server: ");
String serverMessage = consoleInput.readLine();
output.println(serverMessage);
} }
} catch (IOException e) {
System.err.println("Error in server: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac ChatServer.java

C:\socket>java ChatServer
Server is listening on port 1234...
Client connected.
Client: hello server!
Server: hii client , how you doing?
Client: i am good , what about you
Server: i am good too , to end this conversation , write exit
Client disconnected.
```

//Client side

```
import java.net.*;
import java.io.*;
public class ChatClient {
public static void main(String[] args) {
try (Socket socket = new Socket("localhost", 1234)) {
System.out.println("Connected to server.");
BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
BufferedReader consoleInput = new BufferedReader(new InputStreamReader(System.in));
String message;
while (true) {
System.out.print("Client: ");
message = consoleInput.readLine();
output.println(message);
if (message.equalsIgnoreCase("exit")) {
System.out.println("Disconnected from server.");
break;
}
String serverMessage = input.readLine();
System.out.println("Server: " + serverMessage);
}
} catch (IOException e) {
System.err.println("Error in client: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac ChatClient.java

C:\socket>java ChatClient
Connected to server.
Client: hello server!
Server: hii client , how you doing?
Client: i am good , what about you
Server: i am good too , to end this conversation , write exit
Client: exit
Disconnected from server.
```

Q2. Write a java program to accept a filename in client application and check whether it is available on server machine or not, if it there then display its contents on client's terminal otherwise display the message "File Not Found". (Write Client and Server application)

```
//Server side

import java.net.*;
import java.io.*;
public class FileServer {
public static void main(String[] args) {
try (ServerSocket serverSocket = new ServerSocket(1234)) {
System.out.println("Server is listening on port 1234...");
// Accept client connection
try (Socket socket = serverSocket.accept()) {
System.out.println("Client connected.");
// Get input from client
BufferedReader clientInput = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
String filename = clientInput.readLine();
File file = new File(filename);
// Get output stream to send data to client
PrintWriter clientOutput = new PrintWriter(socket.getOutputStream(), true);
if (file.exists() && !file.isDirectory()) {
// If file exists, send its contents
System.out.println("File found: " + filename);
BufferedReader fileReader = new BufferedReader(new FileReader(file));
String line;
while ((line = fileReader.readLine()) != null) {
clientOutput.println(line);
}
fileReader.close();
} else {
// If file doesn't exist, send "File Not Found" message
System.out.println("File not found: " + filename);
clientOutput.println("File Not Found");
}
}
} catch (IOException e) {
System.err.println("Error in server: " + e.getMessage());
}
}
}
```

Output:

```
C:\socket>javac FileServer.java

C:\socket>java FileServer
Server is listening on port 1234...
Client connected.
File found: ServerHi.java
```

```
//Client side
import java.net.*;
import java.io.*;
import java.util.Scanner;
public class FileClient {
public static void main(String[] args) {
try (Socket socket = new Socket("localhost", 1234)) {
System.out.println("Connected to server.");
Scanner scanner = new Scanner(System.in);
System.out.print("Enter the filename: ");
String filename = scanner.nextLine();
PrintWriter clientOutput = new PrintWriter(socket.getOutputStream(), true);
clientOutput.println(filename);
BufferedReader serverInput = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
String response;
while ((response = serverInput.readLine()) != null) {
System.out.println(response);
if (response.equals("File Not Found")) {
break;
} }
} catch (IOException e) {
System.err.println("Error in client: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac FileClient.java

C:\socket>java FileClient
Connected to server.
Enter the filename: ServerH1.java
import java.net.*;
import java.io.*;

public class ServerH1 {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(1234)) {
            System.out.println("Server is listening on port 1234...");
            try (Socket socket = serverSocket.accept()) {
                System.out.println("Client connected.");
                BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                String message = input.readLine();
                System.out.println("Received message: " + message);
            }
        } catch (IOException e) {
            System.err.println("Error in server: " + e.getMessage());
        }
    }
}
```

Q3. Write a java program to accept a number in client application, send it to the server, server will calculates its factors and display them on the client terminal. (Each factor should display after 2 seconds). Write Client and Server application on different terminals.

//Server side

```
import java.net.*;
import java.io.*;
public class FactorServer {
public static void main(String[] args) {
try (ServerSocket serverSocket = new ServerSocket(1234)) {
System.out.println("Server is listening on port 1234...");
// Accept client connection
try (Socket socket = serverSocket.accept()) {
System.out.println("Client connected.");
// Input from client (the number)
BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
int number = Integer.parseInt(input.readLine());
// Output stream to send factors to client
PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
// Calculate factors and send them to the client
System.out.println("Calculating factors of: " + number);
for (int i = 1; i <= number; i++) {
if (number % i == 0) {
output.println(i);
System.out.println("Sent factor: " + i);
try {
Thread.sleep(2000); // 2-second delay between factors
} catch (InterruptedException e) {
e.printStackTrace();
} } }
output.println("End of factors"); // Indicate end of factors
}
} catch (IOException e) {
System.err.println("Error in server: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac FactorServer.java

C:\socket>java FactorServer
Server is listening on port 1234...
Client connected.
Calculating factors of: 28
Sent factor: 1
Sent factor: 2
Sent factor: 4
Sent factor: 7
Sent factor: 14
Sent factor: 28
```

```
//Client side
import java.net.*;
import java.io.*;
import java.util.Scanner;
public class FactorClient {
public static void main(String[] args) {
try (Socket socket = new Socket("localhost", 1234)) {
System.out.println("Connected to server.");
Scanner scanner = new Scanner(System.in);
System.out.print("Enter a number: ");
int number = scanner.nextInt();
PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
output.println(number);
BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
String response;
while ((response = input.readLine()) != null) {
if (response.equals("End of factors")) {
break; // End of factors
}
System.out.println("Received factor: " + response);
}
} catch (IOException e) {
System.err.println("Error in client: " + e.getMessage());
} } }
```

Output:

```
C:\socket>javac FactorClient.java

C:\socket>java FactorClient
Connected to server.
Enter a number: 28
Received factor: 1
Received factor: 2
Received factor: 4
Received factor: 7
Received factor: 14
Received factor: 28
```

Set c:

Q. 1. Write a java program for chatting application. (Use Swing for GUI). (Write Client and Server application on different terminals).

```
//Server side
import java.io.*;
import java.net.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class ChatServerGUI {
private static JTextArea area;
private static JTextField textField;
private static PrintWriter writer;
public static void main(String[] args) {
JFrame frame = new JFrame("Chat Server");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 500);
area = new JTextArea();
area.setEditable(false);
JScrollPane scroll = new JScrollPane(area);
frame.add(scroll, BorderLayout.CENTER);
textField = new JTextField();
frame.add(textField, BorderLayout.SOUTH);
textField.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
sendMessage();
}
});
frame.setVisible(true);
try {
ServerSocket serverSocket = new ServerSocket(12345);
area.append("Server started. Waiting for clients...\n");
Socket clientSocket = serverSocket.accept();
area.append("Client connected: " + clientSocket.getInetAddress().getHostName() + "\n");
writer = new PrintWriter(clientSocket.getOutputStream(), true);
BufferedReader reader = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
String message;
while ((message = reader.readLine()) != null) {
area.append("Client: " + message + "\n");
}
} catch (IOException e) {
area.append("Error: " + e.getMessage());
} }
private static void sendMessage() {
String message = textField.getText();
area.append("Server: " + message + "\n");
writer.println(message);
textField.setText("");
```

} }
Output:

```
//Client side

import java.io.*;
import java.net.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class ChatClientGUI {
private static JTextArea area;
private static JTextField textField;
private static PrintWriter writer;
public static void main(String[] args) {
JFrame frame = new JFrame("Chat Client");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 500);
area = new JTextArea();
area.setEditable(false);
JScrollPane scroll = new JScrollPane(area);
frame.add(scroll, BorderLayout.CENTER);
textField = new JTextField();
frame.add(textField, BorderLayout.SOUTH);
textField.addActionListener(new ActionListener() {
@Override
public void actionPerformed(ActionEvent e) {
sendMessage();
}
});
frame.setVisible(true);
try {
Socket socket = new Socket("localhost", 12345);
writer = new PrintWriter(socket.getOutputStream(), true);
BufferedReader reader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
// Listen for messages from the server
String message;
while ((message = reader.readLine()) != null) {
```

```
area.append("Server: " + message + "\n");
}
} catch (IOException e) {
area.append("Error: " + e.getMessage());
} }
private static void sendMessage() {
String message = textField.getText();
area.append("Client: " + message + "\n");
writer.println(message);
textField.setText("");
}
}
```

Output:

Q2.Write a java program to send and receive the messages from multiple terminals.

//Server side

```
import java.io.*;
import java.net.*;
import java.util.*;
  public class MultiServer {
  private static final int PORT = 12345;
  private static Set<PrintWriter> clientWriters = new HashSet<>();
  public static void main(String[] args) {
    System.out.println("Server started...");
    try (ServerSocket serverSocket = new ServerSocket(PORT)) {
while (true) {
new ClientHandler(serverSocket.accept()).start();
}
} catch (IOException e) {
e.printStackTrace();
} }
private static class ClientHandler extends Thread {
private Socket socket;
private PrintWriter out;
private BufferedReader in;
public ClientHandler(Socket socket) {
this.socket = socket;
}
public void run() {
try {
in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
out = new PrintWriter(socket.getOutputStream(), true);
synchronized (clientWriters) {
clientWriters.add(out);
}
String message;
while ((message = in.readLine()) != null) {
System.out.println("Received: " + message);
synchronized (clientWriters) {
for (PrintWriter writer : clientWriters) {
writer.println(message); // broadcast the message to all clients
} } }
} catch (IOException e) {
e.printStackTrace();
} finally {
try {
socket.close();
} catch (IOException e) {
e.printStackTrace();
}
synchronized (clientWriters) {
clientWriters.remove(out);
```

}}}}}
Output:

```
C:\socket>javac MultiServer.java

C:\socket>java MultiServer
Server started...
Received: Hello All....
Received: good afternoon everyone
Received: out
Received: i support bjp
Received: i support congress
```

```
//Client side
import java.io.*;
import java.net.*;
public class MultiClient {
private static final String SERVER_ADDRESS = "localhost";
private static final int SERVER_PORT = 12345;
public static void main(String[] args) {
try (Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
BufferedReader userInput = new BufferedReader(new InputStreamReader(System.in));
BufferedReader serverInput = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
PrintWriter serverOutput = new PrintWriter(socket.getOutputStream(), true)) {
// Thread to listen to server messages
new Thread(() -> {
try {
String serverMessage;
while ((serverMessage = serverInput.readLine()) != null) {
System.out.println("Server: " + serverMessage);
}
} catch (IOException e) {
e.printStackTrace();
}
}).start();
// Send messages from user to the server
String userMessage;
while (true) {
System.out.print("Enter message: ");
userMessage = userInput.readLine();
if (userMessage.equalsIgnoreCase("exit")) {
break;
}
serverOutput.println(userMessage); // send message to server
}
} catch (IOException e) {
e.printStackTrace();
}
}
}
```

Output:

```
C:\socket>javac MultiClient.java

C:\socket>java MultiClient
Enter message: Hello All....
Enter message: Server: Hello All....
Server: good afternoon everyone
Server: out
i support bjp
Enter message: Server: i support bjp
Server: i support congress
```

```
C:\socket>javac MultiClient.java

C:\socket>java MultiClient
Enter message: good afternoon everyone
Enter message: Server: good afternoon everyone
out
Enter message: Server: out
Server: i support bjp
i support congress
Enter message: Server: i support congress
```

Q3.Write a java program to share a file on server machine between multiple clients.

```
//Server side
import java.io.*;
import java.net.*;
import java.util.concurrent.*;

  public class UploadFileServer {
   private static final int PORT = 12345;
   private static final String FILE_PATH = "C:\socket\bhidu.txt"; // Path to the file to be shared
   public static void main(String[] args) {
     System.out.println("File server started...");
     try (ServerSocket serverSocket = new ServerSocket(PORT)) {
    ExecutorService executor = Executors.newFixedThreadPool(10); // Thread pool for handling
    clients
while (true) {
Socket clientSocket = serverSocket.accept(); // Accept client connections
executor.submit(new ClientHandler(clientSocket)); // Handle the client in a separate thread
}
} catch (IOException e) {
e.printStackTrace();
}
}
  private static class ClientHandler implements Runnable {
  private Socket socket;
  public ClientHandler(Socket socket) {
  this.socket = socket;
}
@Override
public void run() {
try (BufferedInputStream fileInputStream = new BufferedInputStream(new
FileInputStream(FILE_PATH));
DataOutputStream out = new DataOutputStream(socket.getOutputStream())) {
File file = new File(FILE_PATH);
out.writeUTF(file.getName()); // Send file name
out.writeLong(file.length()); // Send file length
byte[] buffer = new byte[4096]; // Buffer to send file in chunks
int bytesRead;
while ((bytesRead = fileInputStream.read(buffer)) != -1) {
out.write(buffer, 0, bytesRead); // Write data to client
}
System.out.println("File sent to client: " + socket.getInetAddress());
} catch (IOException e) {
e.printStackTrace();
} finally {
try {
socket.close();
} catch (IOException e) {
```

```
e.printStackTrace();
}}}}}
```

Output:

```
C:\socket>javac UploadFileServer.java

C:\socket>java UploadFileServer
File server started...
File sent to client: /127.0.0.1
File sent to client: /127.0.0.1
```

```
//Client side

import java.io.*;
import java.net.*;

  public class UploadFileClient {
  private static final String SERVER_ADDRESS = "localhost"; // IP address of the server
  private static final int SERVER_PORT = 12345; // Port number to connect to
  private static final String DOWNLOAD_DIR = "downloads"; // Directory to save the downloaded file
  public static void main(String[] args) {
    try (Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
      DataInputStream in = new DataInputStream(socket.getInputStream())) {
// Receive file name and length from the server
String fileName = in.readUTF();
long fileLength = in.readLong();
// Create the directory to save the file if it doesn't exist
File downloadDir = new File(DOWNLOAD_DIR);
if (!downloadDir.exists()) {
downloadDir.mkdir();
}
// Prepare file output stream to save the file
try (FileOutputStream fileOutputStream = new FileOutputStream(new File(downloadDir,
fileName));
BufferedOutputStream bufferedOutputStream = new
BufferedOutputStream(fileOutputStream)) {
// Read and save the file content
byte[] buffer = new byte[4096];
int bytesRead;
long totalBytesRead = 0;
while (totalBytesRead < fileLength && (bytesRead = in.read(buffer)) != -1) {
bufferedOutputStream.write(buffer, 0, bytesRead);
totalBytesRead += bytesRead;
}
System.out.println("File downloaded successfully: " + fileName);
} catch (IOException e) {
e.printStackTrace();
}
```

```
} catch (IOException e) {
e.printStackTrace();
}
}
}
```

Output:

```
C:\socket>javac UploadFileClient.java

C:\socket>java UploadFileClient
File downloaded successfully: bhidu.txt
```